Per-case forensic report (PDF)

A reviewer (L1 or L2) of the case's company can export a standalone, printable PDF describing everything the system found about one case — what was checked, what was flagged, why the score is what it is, who did what, and how far to trust it. It is designed to be read by someone who has **not** used the application.

Trigger: `POST /cases/{case_id}/reports` (button *Export report* on the case detail page, `CaseReportExport`). **History:** `GET /cases/{case_id}/reports`. Reviewers only; a platform admin can list a company's reports (read-only, audited) but cannot generate one. **Code:** `services/case_report_service.py` (orchestration + storage), `services/case_report_data.py` (gathering — `collect_report_data → ReportData`), `services/case_report_pdf.py` (rendering — `build_report_pdf(ReportData) → bytes`, pure, no DB or network).

How it is generated

Generation is **synchronous** (a few seconds for a typical case; there is no Celery job because the codebase has no async-export pattern; move `generate_case_report` into a task if reports get heavy).

1. `collect_report_data` **reads** what the pipeline already stored: `document_checks`, `case_risk_assessments`, `cross_document_findings`, `case_actions`, `signature_matches` and the append-only `audit_log`. It **never re-runs a check, re-scores a case, regenerates a reason (the reason text is the one frozen at scoring time) or writes its own audit trail.**
2. It downloads each **original** from Blob Storage purely to (a) **re-hash** it, proving the chain of custody still matches the recorded SHA-256, and (b) render the pages that carry findings. The original is only ever read.
3. `build_report_pdf` builds the PDF in memory with PyMuPDF (HTML "story" pages + annotated page renders).
4. The PDF is uploaded to `companies/{company_id}/cases/{case_id}/reports/{report_id}.pdf`, and a `case_reports` row is inserted with `report_sha256`, `page_count`, `file_size_bytes` and the `risk_assessment_id` it reflects; the company's `files_stored` / `storage_bytes` usage counters go up in the same transaction.
5. Audit event `case_report_generated {report_id, page_count, risk_assessment_id, report_sha256}`.

Every generation adds a **new** row; earlier reports stay downloadable. A report is a point-in-time record: it reflects the evidence, assessment and rule versions as they were then, so two reports for the same case can legitimately differ. Storage failure is a 502 and nothing is recorded.

Live overlays vs. the report's annotated pages

These are different mechanisms, on purpose. The live UI draws boxes **on top of** a rendered page in the browser and stores nothing. The report **burns** findings into *its own* rendered page images. That does not violate the immutable-original rule: the original blob is only read, and the annotated renders exist only inside the new report. Both use the same colour convention (red = ELA, orange = copy-move, dashed blue = model judgment, purple = rule-based field exception).

The nine sections

#	Section	What it contains	Data source (`ReportData` field → tables)
1	**Case Details**	Case number, type, status, submitter, submission time, document list and types, who generated the report and when (with their role label), the recorded decision / escalation, each "by Name (Reviewer L1/L2)" from `case_actions.actor_role`	`case_*` scalars, `documents`, `decision: DecisionInfo`, `escalation: EscalationInfo` ← `cases`, `case_actions`, `users`

#	Section	What it contains	Data source (`ReportData` field → tables)
2	**Executive Summary**	Short: tier and score, the decision, how many documents/exceptions, pointers to Sections 3, 4, 5 and 9. Notes when the case had not finished analysis (`pipeline_pending`)	`risk: RiskSummary` , `pipeline_pending` , `exceptions` ← `case_risk_assessments` , `pipeline_status`
3	**Explainable Findings**	Each triggered risk reason: severity, weight, check, document, page, the frozen reason text, and region IDs (R1, R2 …)	`findings: list[FindingRow]` ← `case_risk_assessments.triggered_reasons`
4	**All Checks**	The complete checklist: every check on every document with its result and a one-line summary, the cross-document table, and the signature comparison	`documents[].check_rows: CheckRow` , `cross_document: list[CrossRow]` (None if < 2 documents) ← `document_checks` , `cross_document_findings` , `signature_matches`
5	**Exceptions**	Everything that did **not** pass, written as concrete items ("Total amount 197,500 does not match subtotal + tax"), each linked to the highlighted region that shows it	`exceptions: list[ExceptionItem]` derived from Section 4's data (failed sub-checks, flagged findings, cross-document mismatches, missing required fields, signature results other than *consistent*)
6	**Limitations and Assumptions**	What the report is and is not (system-assisted analysis to support human review, not a certified forensic opinion); which checks are deterministic vs. model judgment; **which checks actually ran on this case (coverage)**; the rule versions and thresholds behind the score; assumptions specific to this case	`coverage: list[CoverageRow]` , `assumptions` , `risk.rule_versions` , `risk.thresholds`
7	**Audit Trail**	Chronological events for the case, **oldest first, capped at the first 150**; any later events are counted ("N later event(s) omitted"). Human actors are shown as "Name (Reviewer L1)" etc., using the role recorded in the event at the time (falling back to the user's current role for older events). Platform-admin access rows are never included (they are platform-only)	`audit: list[AuditRow]` , `audit_truncated` ← `audit_log`
8	**Appendix**	**8.1** extracted fields per document with confidence and whether the field was located; **8.2** SHA-256 of each original with a live **verification** status (`match` / `mismatch` / `unverified` — the original could not be fetched); **8.3** technical parameters of each check	`documents[].fields` , `documents[].hash: HashVerification` , `technical: list[TechnicalBlock]`
9	**PDF Highlighted Regions**	One consolidated page per flagged document page: the original page render with every highlight burned in, a legend, and the list of findings on it	`documents[].annotations: list[PageAnnotation]` + a re-render of the original page

Region IDs

Every highlight gets a report-wide ID **R1, R2, …** (assigned in document order, then page order by `assign_region_ids`). Sections 3 and 5 cite them, and each citation is an internal **PDF link** to the Section 9 page that shows it. Annotation kinds map to the colours above (`AnnotationKind`).

Wording rules

The report follows the same restraint as the rest of the product: model-judgment items are labelled "approximate", the AI-generation and signature results are described as advisory/experimental, and the limitations section states plainly that **the absence of a flagged finding does not guarantee a document is genuine**. It never uses "verified" for a signature.

Output record

`case_reports` row / `CaseReportResponse` (`schemas/case_report.py`):

```
{"id": "uuid", "case_id": "uuid", "generated_by_name": "Seed Admin",
 "generated_at": "2026-09-20T18:02:11Z", "file_size_bytes": 412331, "page_count": 14,
 "report_sha256": "…64 hex…", "risk_tier": "high", "risk_score": 75,
 "download_url": "https://…?sv=…&sig=…"}      // short-lived SAS URL, minted on each list/create call
```

Known limitations

- Only the **first 150 audit events** (oldest first) appear in Section 7; on a long-lived case the *later* events — often the reviewer's decision and the report itself — are omitted from the excerpt and only counted. The full history is in the audit log (`GET /cases/{id}/audit-log`).
- A report of a case whose original is missing or unreadable degrades gracefully: that document's Section 9 page shows a banner explaining the problem, and 8.2 reports `unverified` instead of failing the export.
- Annotated pages exist only for pages that carry findings and whose original could be rendered — PDFs only.
- Generation cost grows with the number of documents and flagged pages, since it is synchronous.
- The report reflects the **latest assessment at generation time**; a case that is re-scored afterwards (for example a signature comparison completing) is not reflected until a new report is generated.